



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

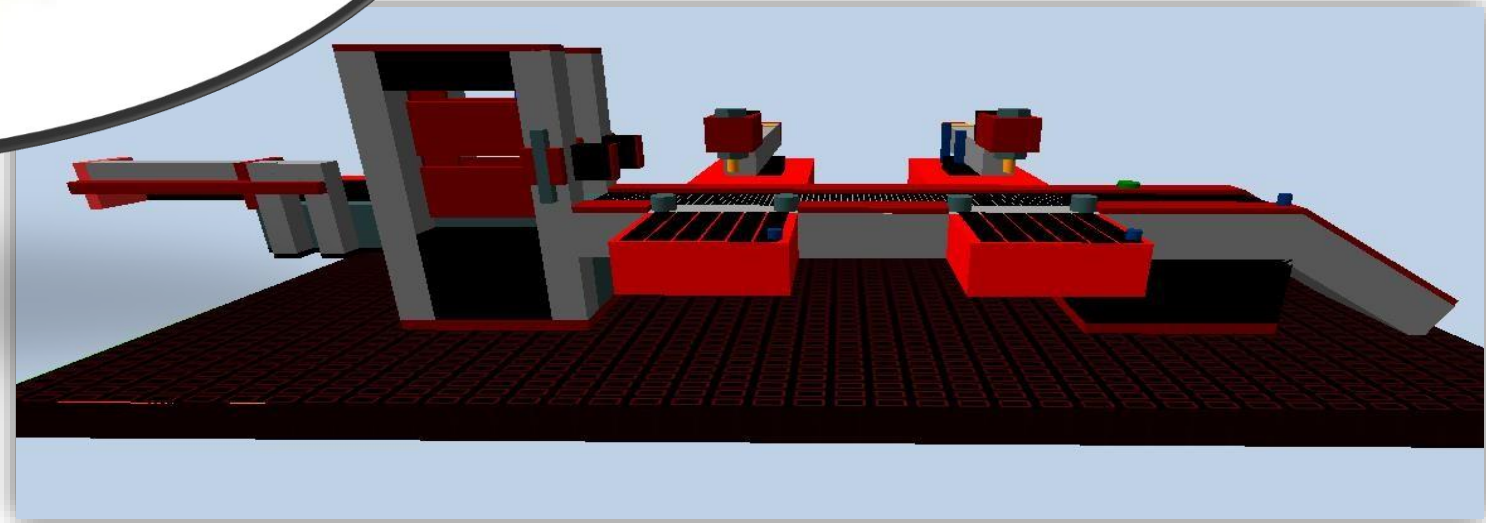
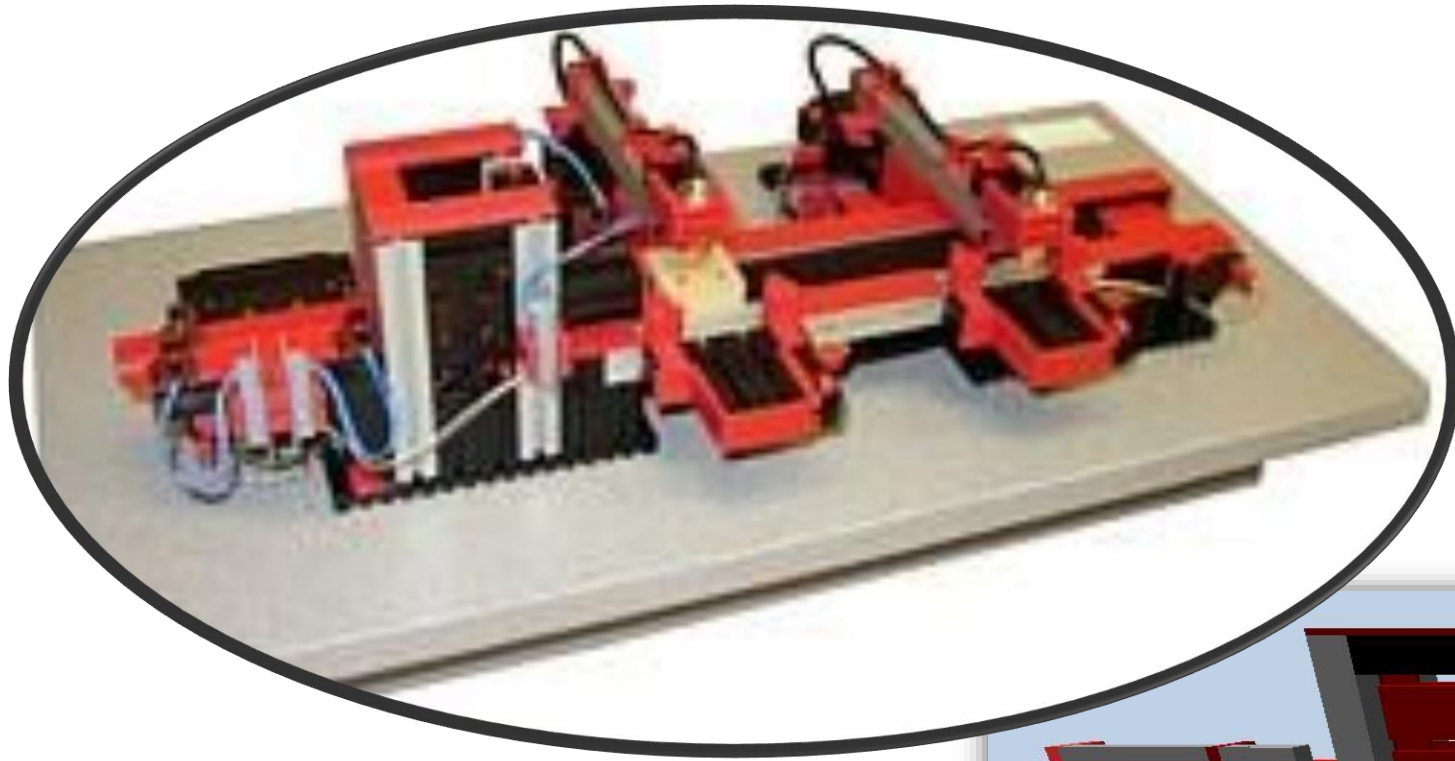
LAB #8

LABWORK 2:

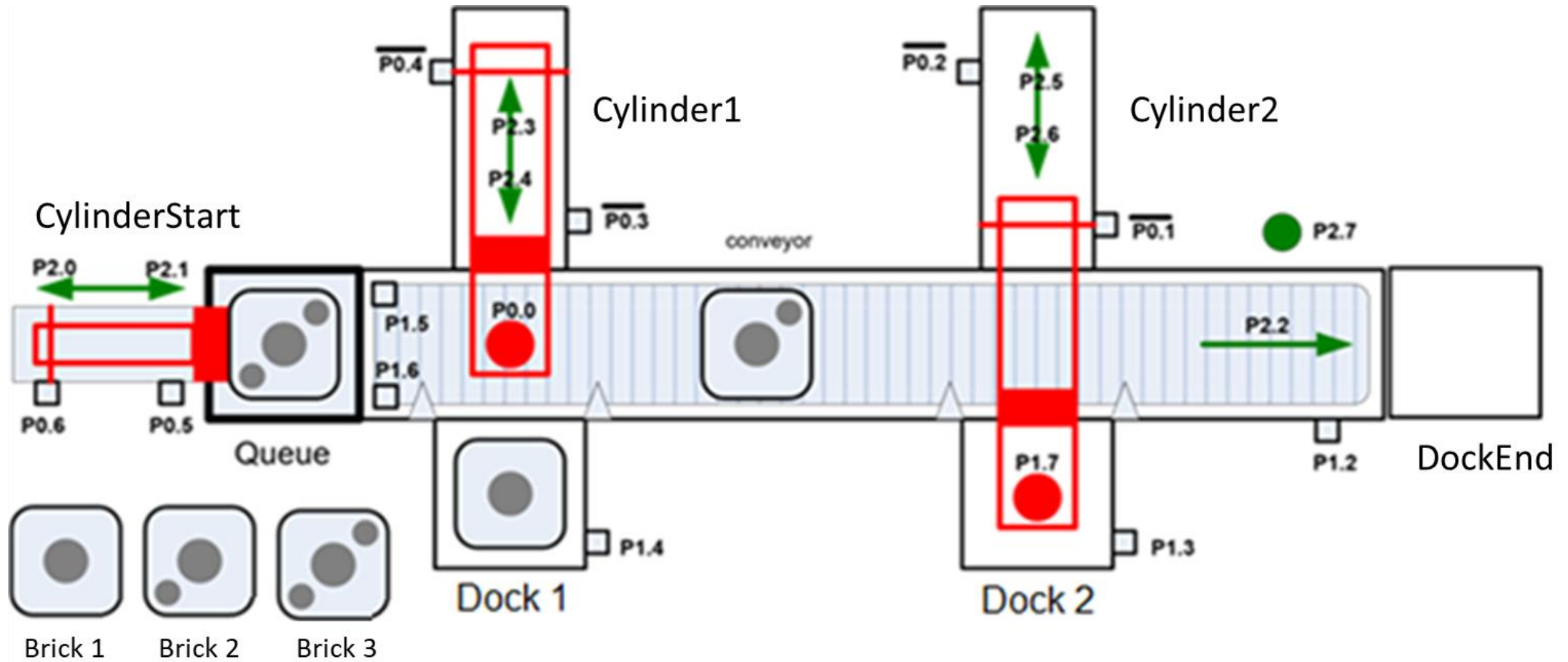
Java Real-Time – synchronization and
communication mechanisms

2022 - 2023

- ☐ Some organization structure for Labwork2
 - ☐ Class Cylinder Interface and Classes Cylinder1 & Cylinder2 & CylinderStart)
 - ☐ Class Mechanism
- ☐ Why Threads?
- ☐ Threads – some aspects to consider
- ☐ Identification of needed Java Threads for Lab2
- ☐ Thread **CylinderThread**
- ☐ Thread **BrickIdentificationThread**
- ☐ Short-lived Threads
- ☐ Long-lived Threads
- ☐ Synchronization with a Semaphore
- ☐ Blocking queue example



LABWORK 2



Some organization structure...

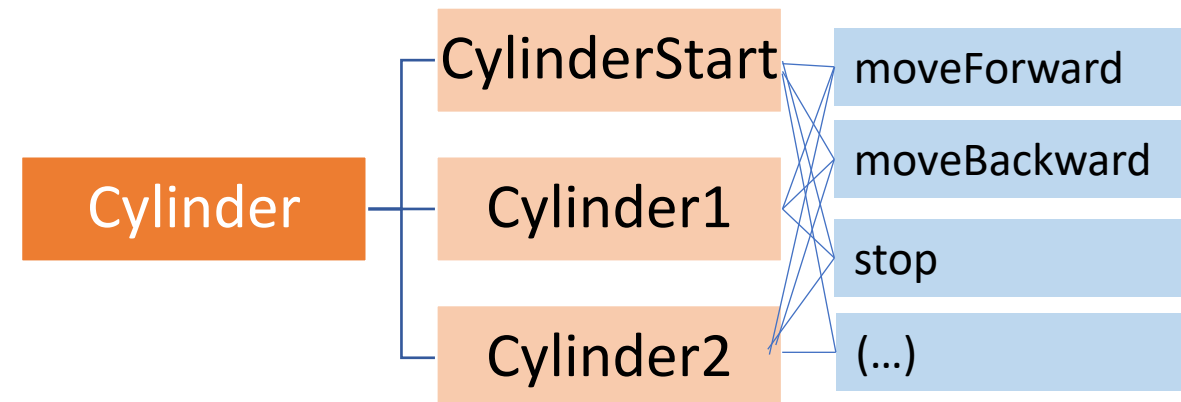
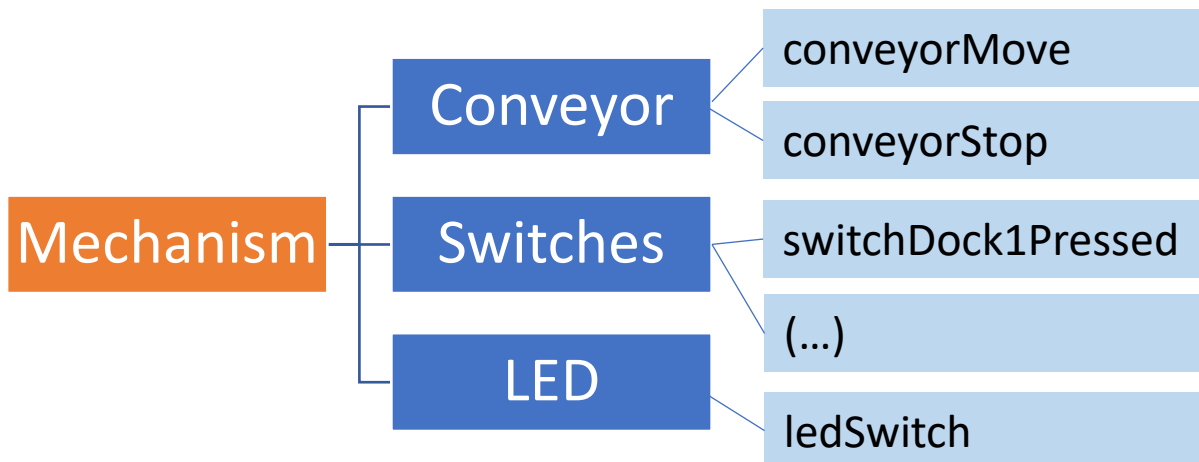
Sensors and actuators: the Java side of JNI



Having the class `SplitterConveyor.java` already developed (the DLL), at the Java side we need to organize our classes into a more logical and straightforward way.

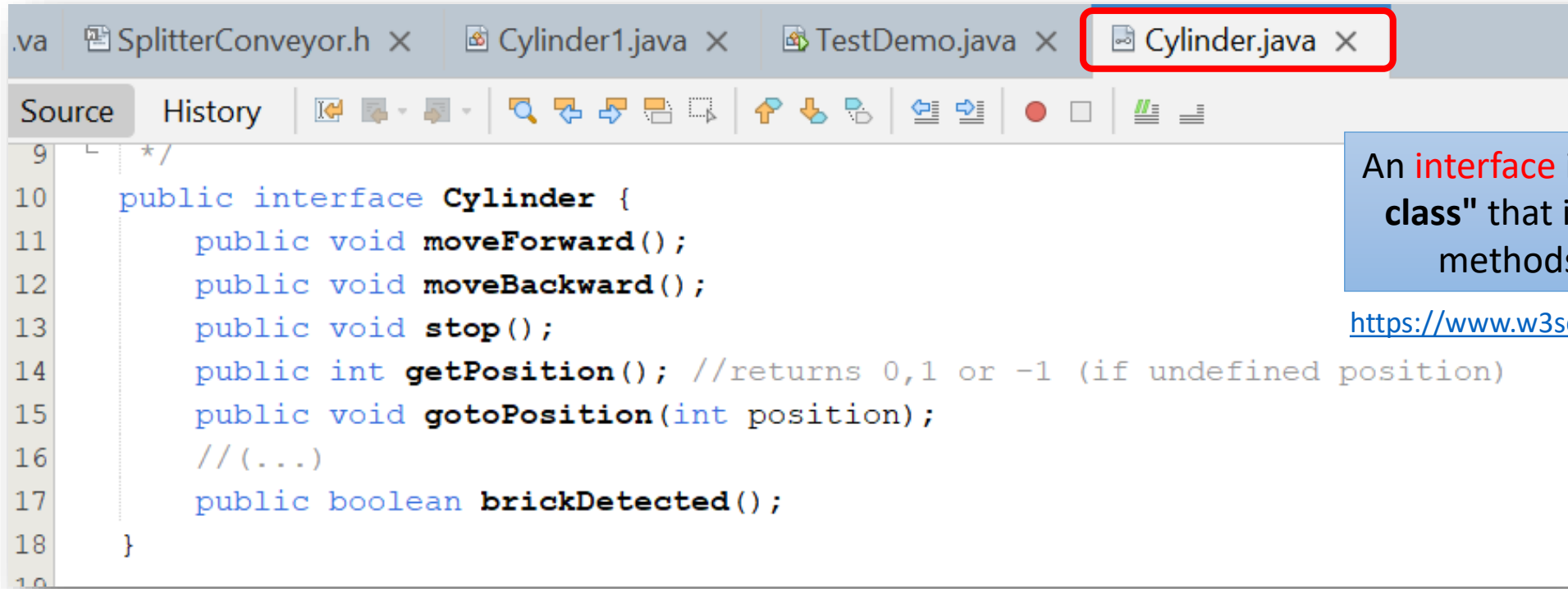
This should be done by aggregating the splitter functionality as several well identified classes, some of them as Java threads.

For instance, we can define concrete **classes for cylinders** operation and a **class named mechanism**, which aggregates the functionality for the conveyor movement, the switches, and the LED.



The Cylinder Interface

Let's start by providing an interface definition for the Cylinders: Java interface -> **Cylinder.java**



The screenshot shows an IDE with several open files: SplitterConveyor.h, Cylinder1.java, TestDemo.java, and Cylinder.java. The Cylinder.java file is selected and highlighted with a red box. The code in the editor defines a public interface named Cylinder with the following methods:

```
9  /*
10 public interface Cylinder {
11     public void moveForward();
12     public void moveBackward();
13     public void stop();
14     public int getPosition(); //returns 0,1 or -1 (if undefined position)
15     public void gotoPosition(int position);
16     //(...)
17     public boolean brickDetected();
18 }
```

An **interface** is a completely "**abstract class**" that is used to group related methods with empty bodies

https://www.w3schools.com/java/java_interface.asp

A class that implements a specific interface must implement the methods of the interface.

The interface has several methods that are used in each cylinder.

Although each cylinder has distinct implementation, e.g. cylinder start has "active_high" sensors, in an abstract way they behave similarly..... as such, interface **Cylinder** specifies and describes the behavior of each cylinder in an abstract way.

Some methods in the interface definition might not be needed, just to illustrate the concept.

Class Cylinder1, Cylinder2, and CylinderStart

.va SplitterConveyor.h x TestDemo.java x Cylinder.java x **Cylinder1.java x**

The same for Cylinder2.java and CylinderStart.java

Source History

```
13 public class Cylinder1 implements Cylinder{
14     @Override
15     public void moveForward() {
16         SplitterConveyor.cylinder1MoveForward();
17     }
18     @Override
19     public void moveBackward() {
20         SplitterConveyor.cylinder1MoveBackward();
21     }
22     @Override
23     public void stop() {
24         SplitterConveyor.cylinder1stop();
25     }
26     @Override
27     public int getPosition() {
28         return SplitterConveyor.cylinder1GetPosition();
29     }
30     @Override
31     public void gotoPosition(int position) {
32         throw new UnsupportedOperationException( string: "Not supported yet."); // Generated from
33         //TODO: move from 0 to 1 and vice-versa
34         // this part is developed in JAVA
35     }
36     @Override
37     public boolean brickDetected() {
38         return SplitterConveyor.isBrickAtDock1();
39     }
40 }
```

Each concrete **cylinder** can follow the **Cylinder interface** and implement the concrete functionality of each Cylinder's method.

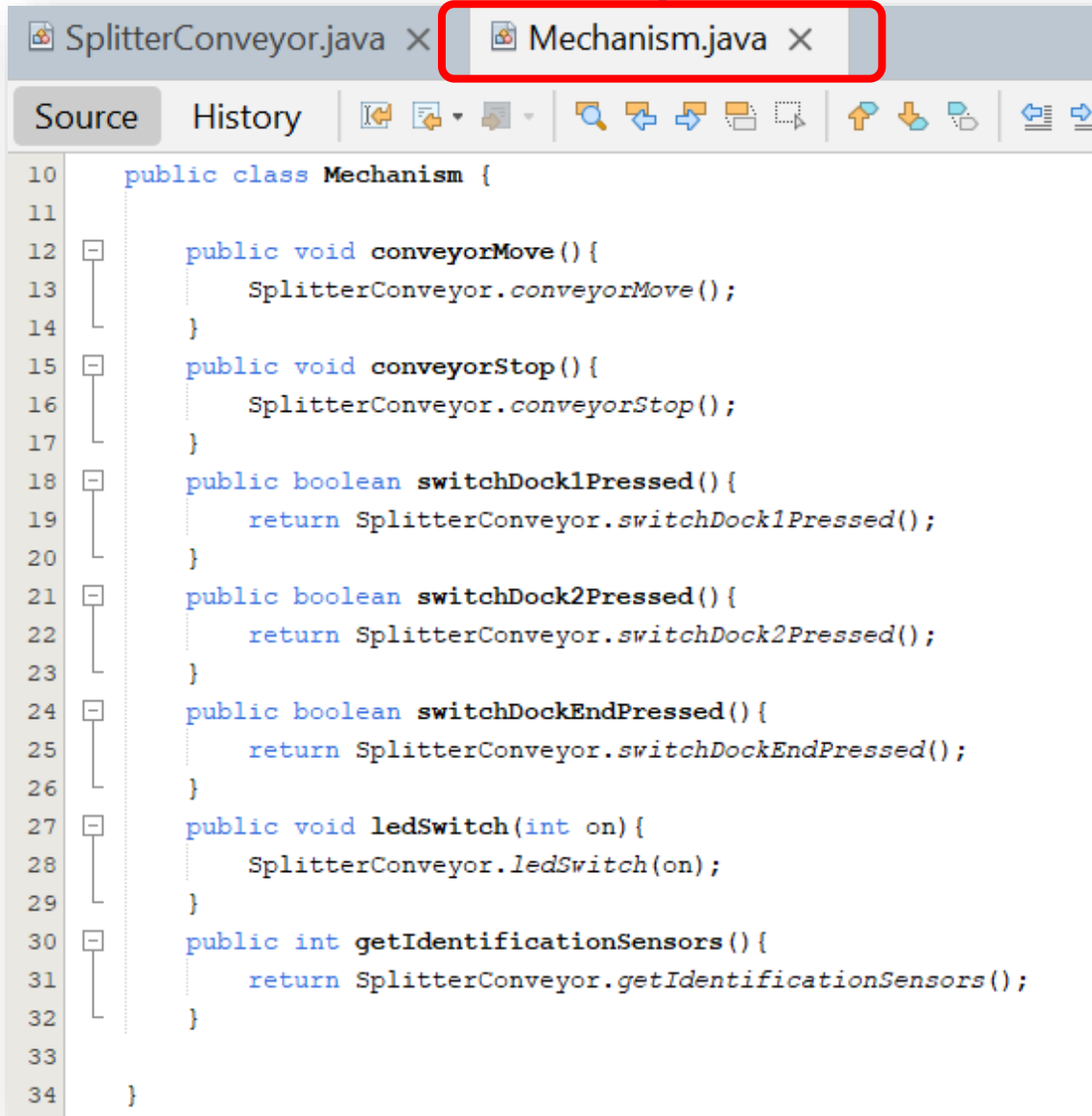
When a class implements a cylinder, it must **implement all** the interface's methods (if not, see the messages from NetBeans for more concrete insight).

Based on the given interface, try develop **three classes** for each **cylinder** existing in the physical system.

Developing class Mechanism



Create a new class **Mechanism.java**



```
10 public class Mechanism {
11
12     public void conveyorMove() {
13         SplitterConveyor.conveyorMove();
14     }
15     public void conveyorStop() {
16         SplitterConveyor.conveyorStop();
17     }
18     public boolean switchDock1Pressed() {
19         return SplitterConveyor.switchDock1Pressed();
20     }
21     public boolean switchDock2Pressed() {
22         return SplitterConveyor.switchDock2Pressed();
23     }
24     public boolean switchDockEndPressed() {
25         return SplitterConveyor.switchDockEndPressed();
26     }
27     public void ledSwitch(int on) {
28         SplitterConveyor.ledSwitch(on);
29     }
30     public int getIdentificationSensors() {
31         return SplitterConveyor.getIdentificationSensors();
32     }
33
34 }
```

The class mechanism deals with:

- the conveyor
- the LED
- the three switches (one at each dock)

Remember, some of these methods might be static, others not

Static methods do not use any instance variables of any object of the class they are defined in. Static methods take all the data from parameters and compute something from those parameters, with no reference to variables.

<https://www.tutorialspoint.com/Java-static-method>

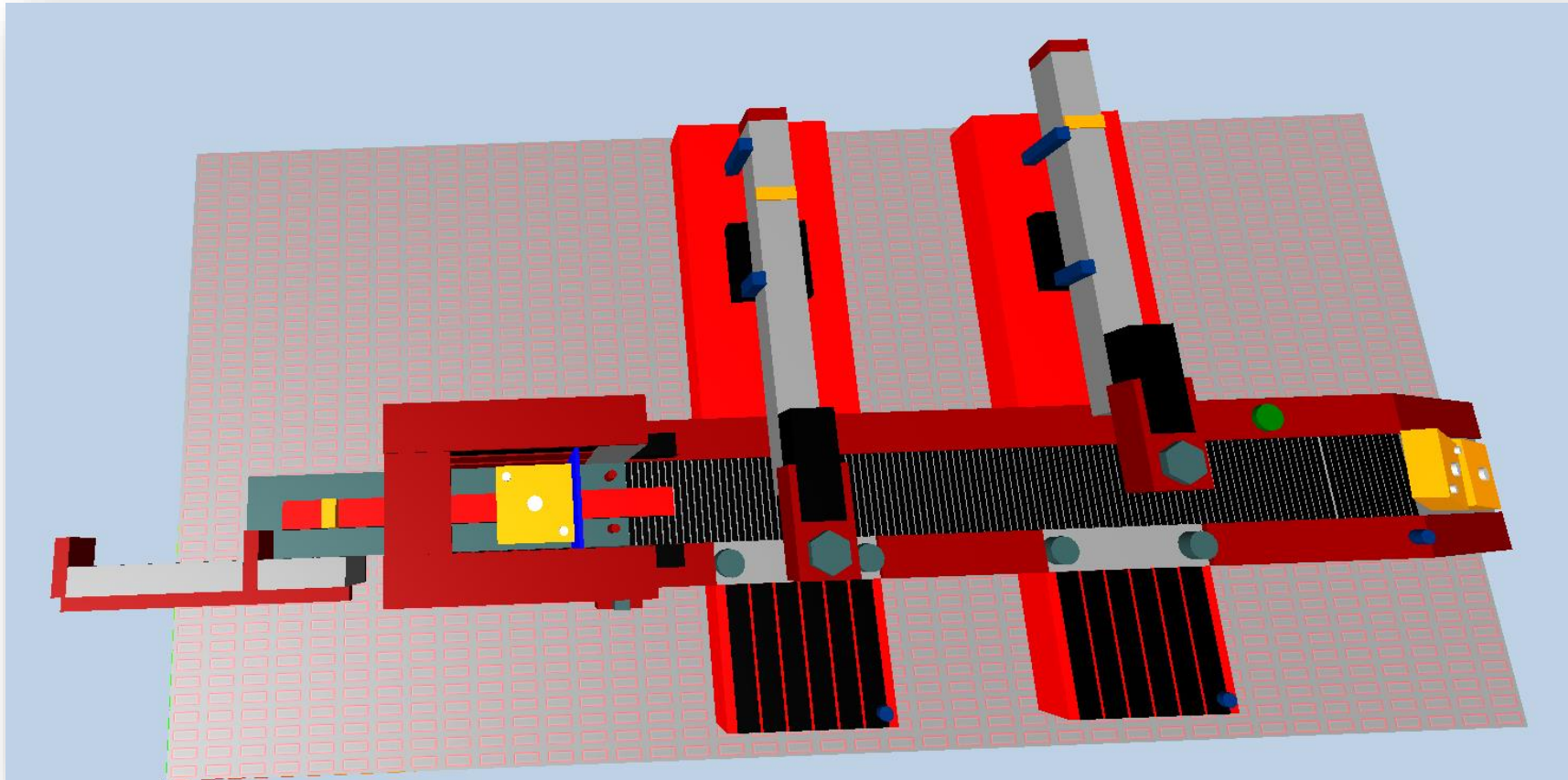
Why Threads?

Aspects to consider

Check the Lab work 2 requirements and elaborate your work “architecture”....

.... which threads should be created to develop the necessary behaviors?

While some are sequential, others take place in parallel?



From previous class.... to study....



Check....

<https://docs.oracle.com/javase/8/docs/api/index.html>

The screenshot shows the Java API documentation for the `Thread` class. The left sidebar lists various packages, with `java.lang` selected. The main content area shows the `Thread` class, which extends `Object` and implements `Runnable`. It includes a description of threads, their priorities, and how to create them. A code snippet shows a subclass `PrimeThread` that extends `Thread` and overrides the `run` method.

The screenshot shows the Java API documentation for the `java.util.concurrent` package. The left sidebar lists various packages, with `java.util.concurrent` selected. The main content area shows the package summary, including a description of utility classes for concurrent programming and a table of interfaces.

Interface	Description
<code>BlockingDeque<E></code>	A <code>Deque</code> that additionally supports blocking operations that wait for the deque to become non-empty when retrieving an element, and wait for space to become available in the deque when storing an element.
<code>BlockingQueue<E></code>	A <code>Queue</code> that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element.
<code>Callable<V></code>	A task that returns a result and may throw an exception.
<code>CompletableFuture.AsyncronousCompletionTask</code>	A marker interface identifying asynchronous tasks produced by async methods.
<code>CompletionService<V></code>	A service that decouples the production of new asynchronous tasks from the consumption of the results of completed tasks.
<code>CompletionStage<T></code>	A stage of a possibly asynchronous computation, that performs an action or computes a value when another <code>CompletionStage</code> completes.
<code>ConcurrentMap<K,V></code>	A <code>Map</code> providing thread safety and atomicity guarantees.
<code>ConcurrentNavigableMap<K,V></code>	A <code>ConcurrentMap</code> supporting <code>NavigableMap</code> operations, and recursively so for its navigable sub-maps.
<code>Delayed</code>	A mix-in style interface for marking objects that should be acted upon after a given delay.
<code>Executor</code>	An object that executes submitted <code>Runnable</code> tasks.
<code>ExecutorService</code>	An <code>Executor</code> that provides methods to manage termination and methods that can produce a <code>Future</code> for tracking progress of one or more asynchronous tasks.
<code>ForkJoinPool.ForkJoinWorkerThreadFactory</code>	Factory for creating new <code>ForkJoinWorkerThreads</code> .
<code>ForkJoinPool.ManagedBlocker</code>	Interface for extending managed parallelism for tasks running in <code>ForkJoinPools</code> .

<https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html>

Threads – some aspects to consider



Two ways to start a thread (1)

```
@FunctionalInterface  
public interface Runnable
```

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. The class must define a method of no arguments called run.

<https://docs.oracle.com/javase/8/docs/api/index.html>

In order to create **threads**, we need to provide a runnable object, as shown in the example.

Any Runnable class must implement the method **run()**.

Notice that HelloRunnable has got this method.

```
public class HelloRunnable implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("Hello from thread!");  
    }  
    public static void main(String[] args) {  
        (new Thread(new HelloRunnable())).start();  
    }  
}
```

https://en.wikipedia.org/wiki/Java_concurrency

Threads – some aspects to consider



Two ways to start a thread (2)

```
public class Thread  
extends Object  
implements Runnable
```

A *thread* is a thread of execution in a program. The Java Virtual Machine allows an application to have multiple threads of execution running concurrently.

Every thread has a priority. Threads with higher priority are executed in preference to threads with lower priority. Each t

<https://docs.oracle.com/javase/8/docs/api/index.html>

In order to create threads, we need to implement a subclass of **Thread**.

```
public class HelloThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("Hello from thread!");  
    }  
    public static void main(String[] args) {  
        (new HelloThread()).start();  
    }  
}
```

https://en.wikipedia.org/wiki/Java_concurrency

Threads – aspects to consider

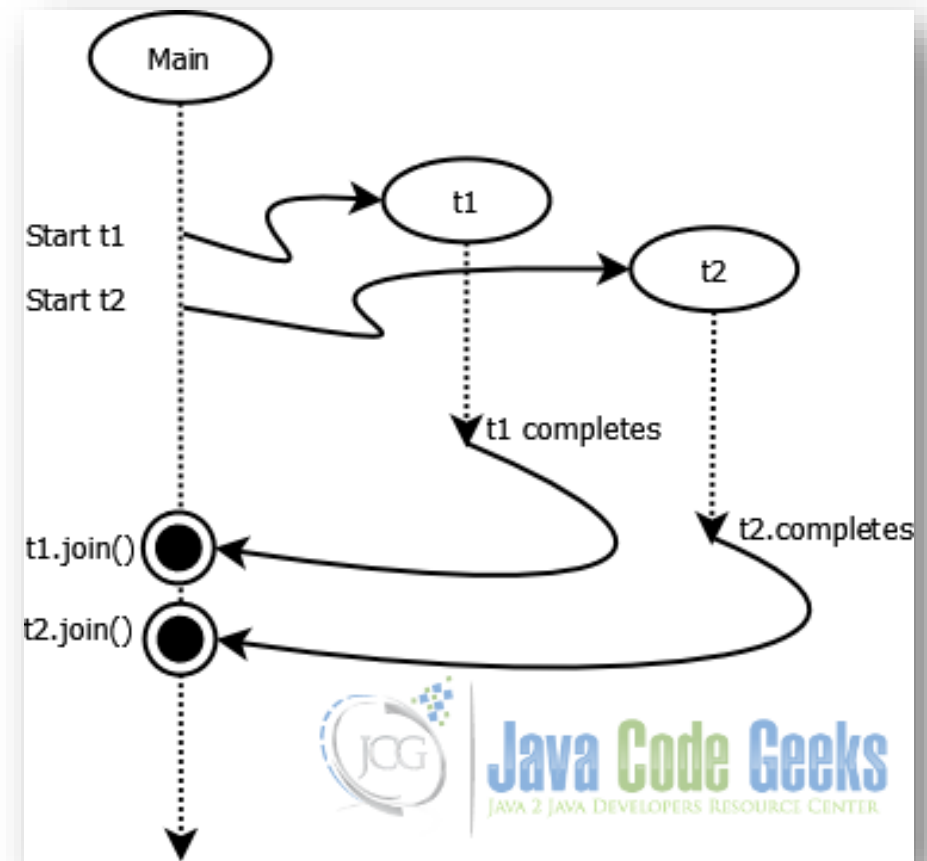
Waiting for threads termination: `Thread.join`

- When **Thread.join** is called, it suspends the execution of the calling thread until the target thread is alive.
- Once the target thread is done with its job and terminates, the caller thread will automatically wake up.
In the diagram, the caller thread starts two threads t1 and t2, each thread is assigned one specific task.
- The caller thread then calls on **t1.join** so that it is blocked from further execution till t1 finishes its job.
Once t1 is done with its execution and terminates, it wakes up the caller thread.
Next, caller thread calls **t2.join** to wait on t2 till it is done with its task.
Once t2 finished, caller thread wakes up and continues in its execution.

`join()`

Waits for this thread to die.

<https://docs.oracle.com/javase/8/docs/api/index.html>



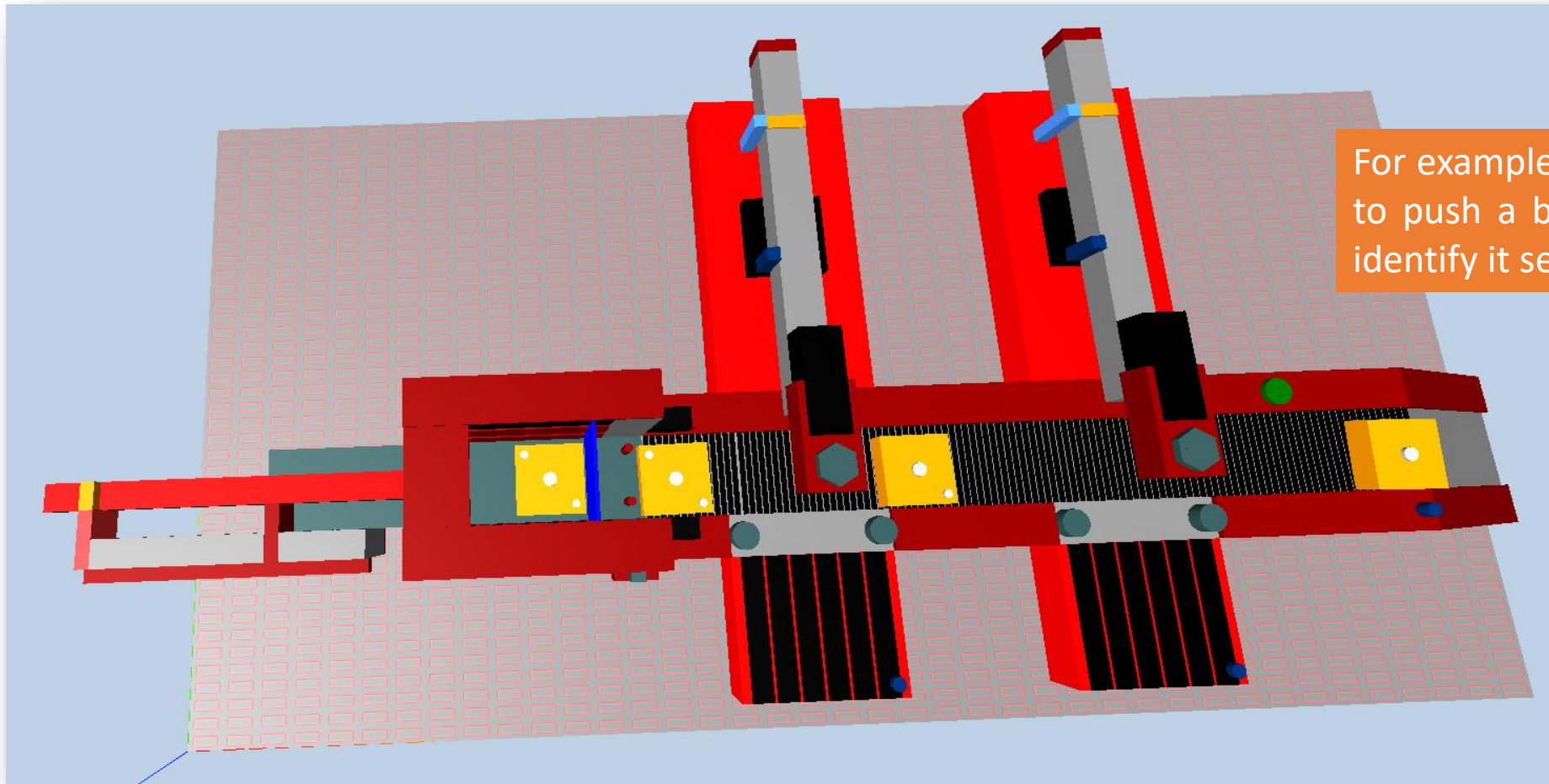
<https://examples.javacodegeeks.com/core-java/threads/java-thread-join-example/>

Identification of needed Java Threads for Lab2

Aspects to consider

Identify which actions / behaviors can occur simultaneously in the **SplitterConveyor**

The question is how to orchestrate their execution...



For example, it is physically not possible to push a brick with cylinder start and identify it sequentially.

Thread CylinderThread



Aspects to consider....

```
12  *author: aro
13  */
14  public class CylinderThread extends Thread{
15
16      private final Cylinder cylinder;
17
18      public CylinderThread (Cylinder cylinder){
19          this.cylinder = cylinder;
20      }
21
22      public void initializeCylinderThread() throws InterruptedException {
23          //if calibrated
24          //goto(1)
25          //goto(0)
26          this.cylinder.gotoPosition( position:1);
27          this.cylinder.gotoPosition( position:0);
28          Thread.sleep( millis: 500);
29      }
30
31      @Override
32      public void run(){
33          try {
34              this.initializeCylinderThread();
35          } catch (InterruptedException ex) {
36              Logger.getLogger( name: CylinderThread.class.getName()).log( level: Level.SEVERE, msg: null, thrown: ex);
37          }
38      }
39
40  }
```

The class **CylinderThread** might be defined as shown on the right).

Its constructor accepts an object that implements the interface Cylinder.

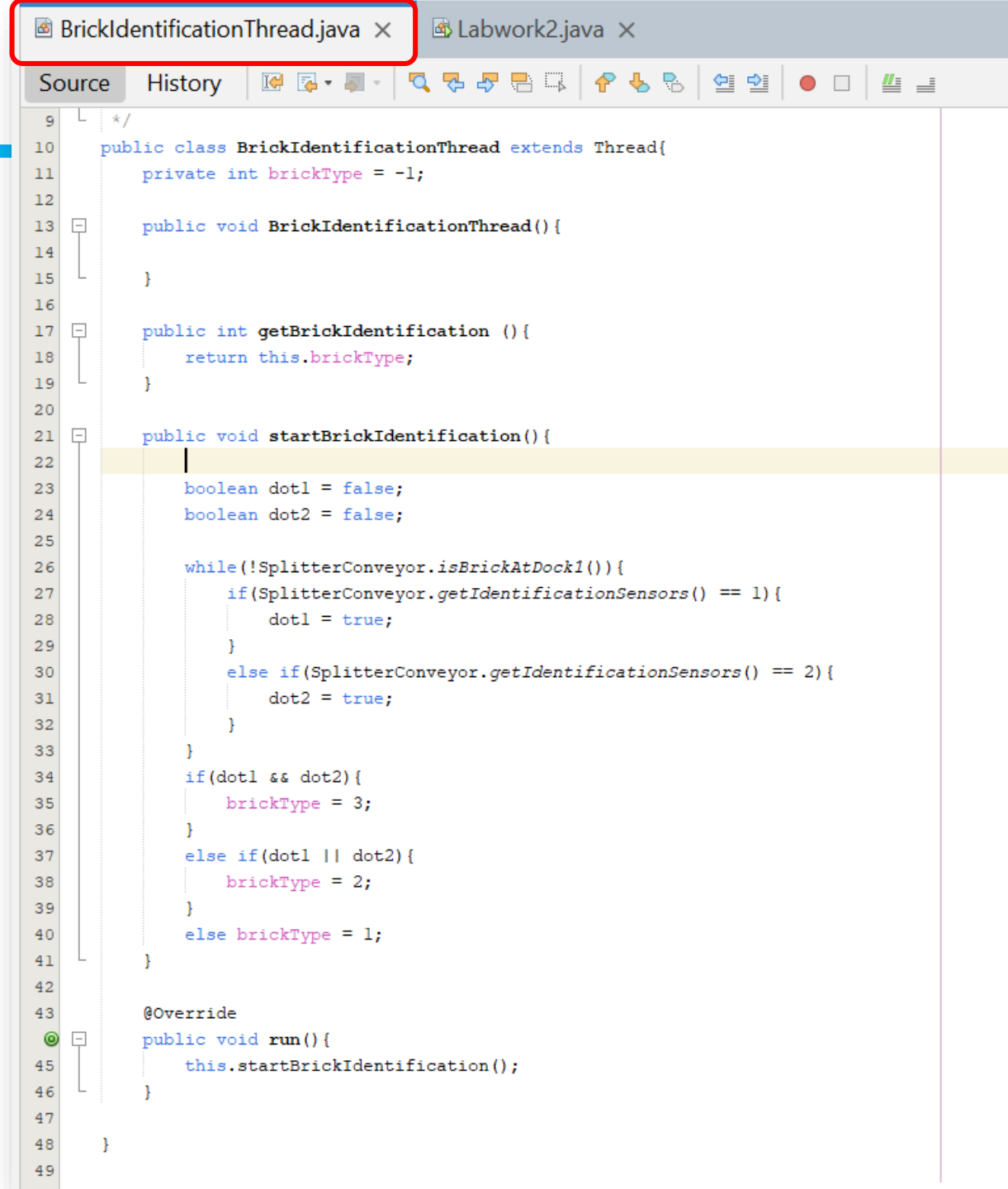
This means that **CylinderThread** can accept and operate with any of the objects instantiated from class Cylinder1, Cylinder2 or Cylinder3.

In this example, regardless of which cylinder provided, after calling the start() method, the **CylinderThread** should sequentially call goto(1) then goto(0) of the provided cylinder.

Thread BrickIdentificationThread

Aspects to consider

BrickIdentificationThread is an example of a thread that you might need to model.



```
BrickIdentificationThread.java X Labwork2.java X
Source History
9  /*
10 public class BrickIdentificationThread extends Thread{
11     private int brickType = -1;
12
13     public void BrickIdentificationThread(){
14
15     }
16
17     public int getBrickIdentification(){
18         return this.brickType;
19     }
20
21     public void startBrickIdentification(){
22
23         boolean dot1 = false;
24         boolean dot2 = false;
25
26         while(!SplitterConveyor.isBrickAtDock1()){
27             if(SplitterConveyor.getIdentificationSensors() == 1){
28                 dot1 = true;
29             }
30             else if(SplitterConveyor.getIdentificationSensors() == 2){
31                 dot2 = true;
32             }
33         }
34         if(dot1 && dot2){
35             brickType = 3;
36         }
37         else if(dot1 || dot2){
38             brickType = 2;
39         }
40         else brickType = 1;
41     }
42
43     @Override
44     public void run(){
45         this.startBrickIdentification();
46     }
47
48 }
49
```

Thread BrickIdentificationThread



Aspects to consider....

To properly use the **BrickIdentificationThread**, it is necessary to coordinate it with **CylinderThread**

```
13  * @param args the command line arguments
14  * @throws java.lang.InterruptedException
15  */
16  public static void main(String[] args) throws InterruptedException {
17      // TODO code application logic here
18      SplitterConveyor.initializeHardwarePorts();
19      Mechanism mechanism = new Mechanism();
20      mechanism.conveyorMove();
21
22      CylinderStart clStart = new CylinderStart();
23      CylinderThread clStartT = new CylinderThread(cylinder: clStart);
24      BrickIdentificationThread brickIDT = new BrickIdentificationThread();
25
26      System.out.println("startting identification process...");
27      brickIDT.start();
28      clStartT.start();
29      clStartT.join();
30      brickIDT.join();
31      int brickID = brickIDT.getBrickIdentification();
32      System.out.print("the result of brick identification is: ");
33      switch (brickID) {
34          case 1: System.out.println("brick 1"); break;
35          case 2: System.out.println("brick 2"); break;
36          case 3: System.out.println("brick 3"); break;
37      }
38
39  }
40
41  }
```

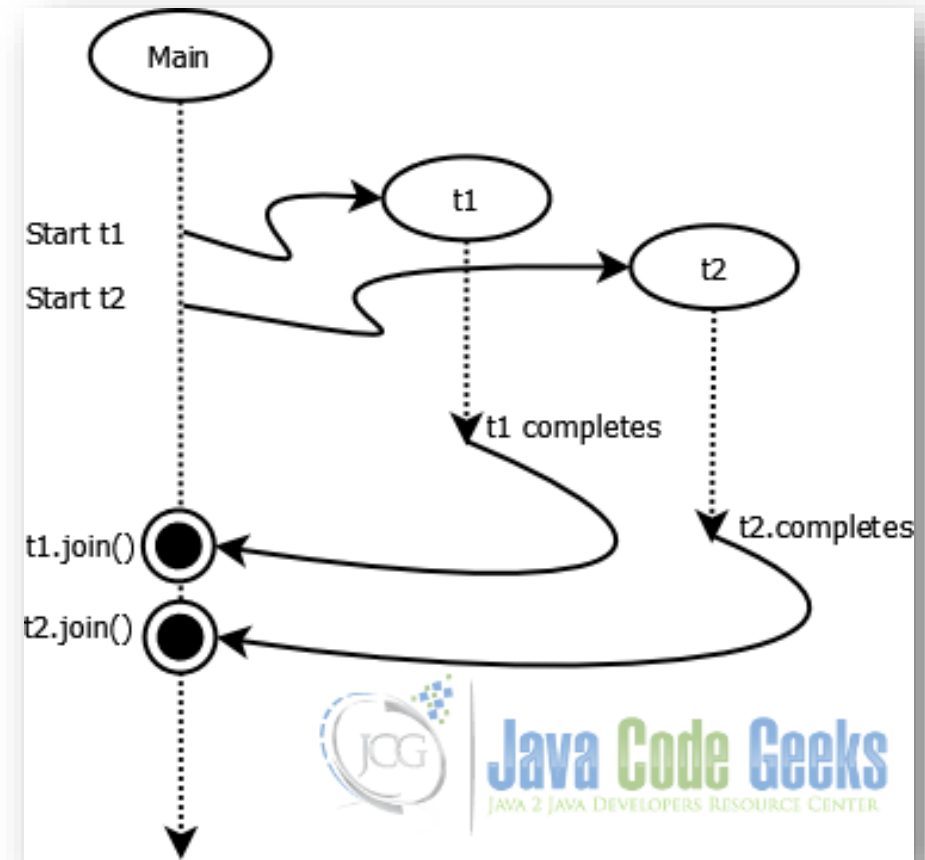
Short-lived Threads

The previous example is a short-lived thread....

Threads which start execution, perform their job in a concurrent way, and complete....

Generally, they perform a single job then finish.

Additionally, we might **synchronize** them (as illustrated in the picture), but not strictly necessary....



Short-lived Threads - examples



Single-thread (implicit MainThread)

```
SortExample.java x
Source History
4 public class SortExample {
5     public static int[] generateHugeArray(int size) {
6         Random rd = new Random(); // creating Random object
7         int[] arr = new int[size];
8         for (int i = 0; i < arr.length; i++) {
9             arr[i] = rd.nextInt(); // storing random integers in an array
10        }
11    }
12 }
13
14
15 public static void main(String[] args) {
16     // Our arr contains 8 elements
17     //int[] arr = {13, 7, 6, 45, 21, 9, 101, 102};
18     int arr1[] = generateHugeArray(2000000);
19     int arr2[] = generateHugeArray(2000000);
20
21     long ti = System.currentTimeMillis();
22     Arrays.sort(arr1);
23     Arrays.sort(arr2);
24     long tf = System.currentTimeMillis();
25
26     //System.out.printf("Modified arr1[] : %s\n", Arrays.toString(arr1));
27     //System.out.printf("Modified arr2[] : %s\n", Arrays.toString(arr2));
28     System.out.printf("Time : %d milliseconds", tf - ti);
29 }
30 }
31
```

SortExample > main >

Output x

Debugger Console x labwork2 (run) x

run:
Time : 378 millisecondsBUILD SUCCESSFUL (total time: 0 seconds)

Multi-thread

```
SortExample.java x
Source History
13
14 public static Thread sortParallel(int []array) {
15     Thread worker = new Thread ( ) {
16         public void run() {
17             Arrays.sort(array);
18         }
19     };
20     worker.start();
21     return worker;
22 }
23
24 public static void main(String[] args) throws InterruptedException {
25     // Our arr contains 8 elements
26     //int[] arr = {13, 7, 6, 45, 21, 9, 101, 102};
27     int arr1[] = generateHugeArray(2000000);
28     int arr2[] = generateHugeArray(2000000);
29
30     long ti = System.currentTimeMillis();
31     Thread worker1 = sortParallel(arr1);
32     Thread worker2 = sortParallel(arr2);
33     worker1.join();
34     worker2.join();
35     long tf = System.currentTimeMillis();
36
37     //System.out.printf("Modified arr1[] : %s\n", Arrays.toString(arr1));
38     //System.out.printf("Modified arr2[] : %s\n", Arrays.toString(arr2));
39     System.out.printf("Available processors : %d\n", Runtime.getRuntime().availableProcessors());
40     System.out.printf("Time : %d milliseconds", tf - ti);
41 }
42 }
```

SortExample > main >

Output - labwork2 (run) x

run:
Available processors : 8
Time : 244 millisecondsBUILD SUCCESSFUL (total time: 0 seconds)

Long-lived Threads



Threads that after being created, stay handling the problems for a long time.



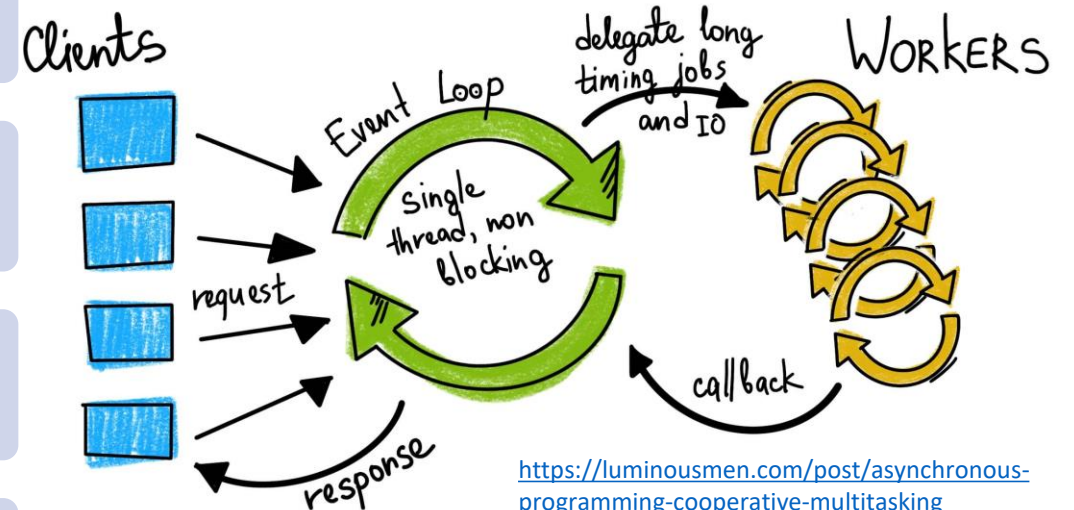
They are structured in a loop-like way, holding-on for a new Job/problem to work on,



...exhibiting multi-tasking cooperative, and reactive, behavior patterns



In many situations, long-lived threads are desirable, to reduce overhead for task creation, memory garbage collection..., less dynamic allocation



Long-lived Threads - example



```
ServerThread.java x
Source History
13  */
14  public class ServerThread extends Thread{
15      private boolean interrupted;
16
17      @Override
18      public boolean isInterrupted() {
19          return interrupted;
20      }
21      public void setInterrupted(boolean interrupted) {
22          this.interrupted = interrupted;
23      }
24
25      @Override
26      public void run() {
27          this.setInterrupted( interrupted:false);
28          while(!interrupted) {
29              //do something
30              System.out.println( x:"doing some task here until interrupted");
31              try {
32                  Thread.sleep( millis:1000);
33              } catch (InterruptedException ex) {
34                  Logger.getLogger( name:ServerThread.class.getName()).log( level:Level.SEVERE, msg:null, thrown:ex);
35              }
36          }
37
38      public static void main(String[] args) throws InterruptedException {
39          ServerThread worker = new ServerThread();
40          worker.start();
41          Thread.sleep( millis:3000);
42          worker.setInterrupted( interrupted:true);
43          System.out.println( x:"Thread was interrupted!");
44      }
45  }
```

Usually in a **while(true) {...} loop** until interrupted

The **interrupted variable** can be set internally (inside the loop), or externally (in other part of the Java application)

Synchronization with a Semaphore



```
ServerThread.java x
Source History
15 public class ServerThread extends Thread{
16     private boolean interrupted;
17     private Semaphore semaphoreSynch = null;
18
19     public ServerThread(){ this.semaphoreSynch = new Semaphore(1:0); }
20     @Override
21     public boolean isInterrupted(){ return interrupted; }
22     public void setInterrupted(boolean interrupted){ this.interrupted = interrupted; }
23
24     public Semaphore getSemaphoreSynch(){ return semaphoreSynch; }
25
26     @Override
27     public void run(){
28         this.setInterrupted( interrupted:false);
29         while(!interrupted){
30             try {
31                 semaphoreSynch.acquire();
32                 System.out.println( x:"doing some task...");
33             } catch (InterruptedException ex) {
34                 Logger.getLogger( name:ServerThread.class.getName()).log( level:Level.SEVERE, msg:null, thrown:ex);
35             }
36         }
37     }
38     public static void main(String[] args) {
39         ServerThread worker = new ServerThread();
40         worker.start();
41         Semaphore semaphore = worker.getSemaphoreSynch();
42
43         semaphore.release();
44         semaphore.release();
45         semaphore.release();
46         worker.setInterrupted( interrupted:true); // it does not work. why?
47         System.out.println( x:"Thread was interrupted!");
48     }
49 }
```

Using the previous example, we can use a Java semaphore to synchronize work.

After three **releases**, the Thread blocks by the semaphore `acquire()`, until more releases.

In order to successfully get out of the loop, it can be reengineered by firstly testing **`availablePermits()>0`** before calling the **`acquire()`** method.

Blocking queue example

```
ServerThread.java × ServerThread2.java ×
Source History
17 public class ServerThread2 extends Thread{
18     private boolean interrupted;
19     private ArrayBlockingQueue<Integer> queue = null;
20
21     public ServerThread2(){ this.queue = new ArrayBlockingQueue(10); }
22     @Override
23     public boolean isInterrupted(){ return interrupted; }
24     public void setInterrupted(boolean interrupted){ this.interrupted = interrupted; }
25
26     public ArrayBlockingQueue getQueue(){ return queue; }
27
28     @Override
29     public void run(){
30         this.setInterrupted( interrupted:false);
31         while(!interrupted){
32             try {
33                 int x = queue.take();
34                 System.out.printf( format:"\ngot %d...", args:x);
35             } catch (InterruptedException ex) {
36                 Logger.getLogger( name:ServerThread.class.getName()).log( level:Level.SEVERE, msg:null, thrown:ex);
37             }
38         }
39     }
40     public static void main(String[] args) {
41         ServerThread2 worker = new ServerThread2();
42         worker.start();
43         ArrayBlockingQueue<Integer> queue1 = worker.getQueue();
44
45         queue1.add( e:100);
46         queue1.add( e:200);
47         queue1.add( e:300);
48         worker.setInterrupted( interrupted:true); // it does not work. why?
49         System.out.println( x:"Thread was interrupted!");
50     }
51 }
```

The example illustrates a utilization of a blocking queue in a thread.

Inside the **run()** method, the thread keeps blocked in the **take()** method until a new message (an integer value) arrives.

Blocking queue – other examples

From the JavaDocs:

- A [ConcurrentLinkedQueue](#) is an appropriate choice when many threads will share access to a common collection. This queue does not permit null elements.
- [ArrayBlockingQueue](#) is a classic "bounded buffer", in which a fixed-sized array holds elements inserted by producers and extracted by consumers. This class supports an optional fairness policy for ordering waiting producer and consumer threads
- [LinkedBlockingQueue](#) typically have higher throughput than array-based queues but less predictable performance in most concurrent applications.

Package java.util.concurrent

Utility classes commonly useful in concurrent programming.

See: Description

Interface	Description
BlockingDeque<E>	A Deque that additionally supports blocking operations that wait for the deque to become non-empty when retrieving an element, and wait for space to become available in the deque when storing an element.
BlockingQueue<E>	A Queue that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element.
Callable<V>	A task that returns a result and may throw an exception.
CompletableFuture , AsynchronousCompletionTask	A marker interface identifying asynchronous tasks produced by async methods.
CompletionService<V>	A service that decouples the production of new asynchronous tasks from the consumption of the results of completed tasks.
CompletionStage<T>	A stage of a possibly asynchronous computation, that performs an action or computes a value when another CompletionStage completes.
ConcurrentMap<K,V>	A Map providing thread safety and atomicity guarantees.
ConcurrentNavigableMap<K,V>	A ConcurrentMap supporting NavigableMap operations, and recursively so for its navigable sub-maps.
Delayed	A mix-in style interface for marking objects that should be acted upon after a given delay.
Executor	An object that executes submitted Runnable tasks.
ExecutorService	An Executor that provides methods to manage termination and methods that can produce a Future for tracking progress of one or more asynchronous tasks.
ForkJoinPool , ForkJoinWorkerThreadFactory	Factory for creating new ForkJoinWorkerThreads .
ForkJoinPool , ManagedBlocker	Interface for extending managed parallelism for tasks running in ForkJoinPools .

<https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html>

Suggestions for work

- Brainstorm your lab work to perceive which threads should be short lived or to persist.
- Could a statistics collection thread be a long-running one?
- Explore the `java.util.concurrent` and decide which concurrent mechanisms can be used to faithfully satisfy the requirements in the lab work.
- However, Keep it simple (but not simplistic).





KEEP UP THE
GOOD
WORK!